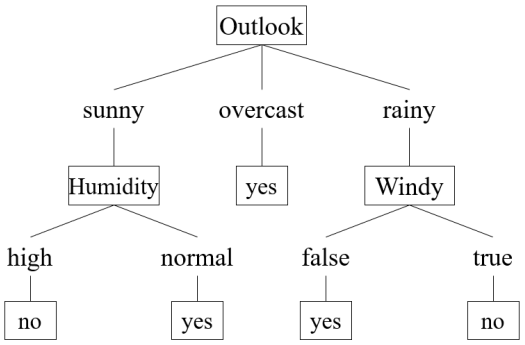
Lecture 2 - Decision Trees

When to use Decision Trees:

- each instance consists of an attribute with discrete values (e.g. sunny, cloudy, etc.)
- the classification is over discrete values (e.g. yes / no)
- it is ok to have disjunctive discriptions (any boolean function can be represented)
- it is ok for the training data to contain errors
- it is ok for the training data to contain missing values

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	Normal	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	High	Strong	Yes
D8	Sunny	Mild	Normal	Weak	No
D9	Sunny	Hot	Normal	Weak	Yes
D10	Rain	Mild	Normal	Strong	Yes
D11	Sunny	Cool	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No



Database → Decision Tree

Entropy

Let S be a sample of training examples, p^+ be the proportion of positive examples in S, and p^- be the proportion of negative examples in S. The **entropy** measures the *impurity* of S.

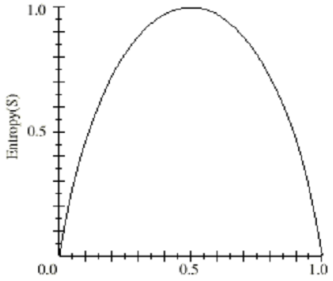
$$E(S) = -p^+ \cdot \log_2 p^+ - p^- \cdot \log_2 p^-$$

Example from the above database:

$$p_{yes} = -\left(\frac{9}{14}\right)\log_2\left(\frac{9}{14}\right) = 0.41$$

$$p_{no} = -\left(\frac{5}{14}\right)\log_2\left(\frac{5}{14}\right) = 0.53$$

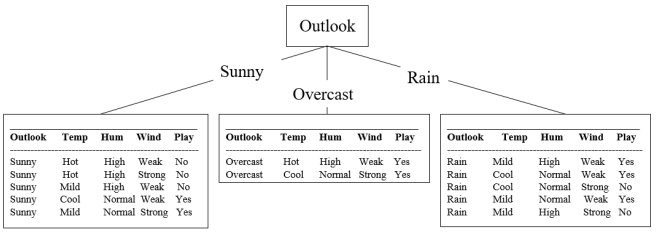
$$E(S) = p_{yes} + p_{no} = 0.94$$



Information Gain

Let S be a sample of training examples, and A be an attribute present in S. The **information gain** is the expected reduction in entropy caused by partitioning S based on attribute A.

$$\text{Gain}(S, A) = E(S) - \sum_{v \in A} \frac{|S_v|}{|S|} E(S_v)$$

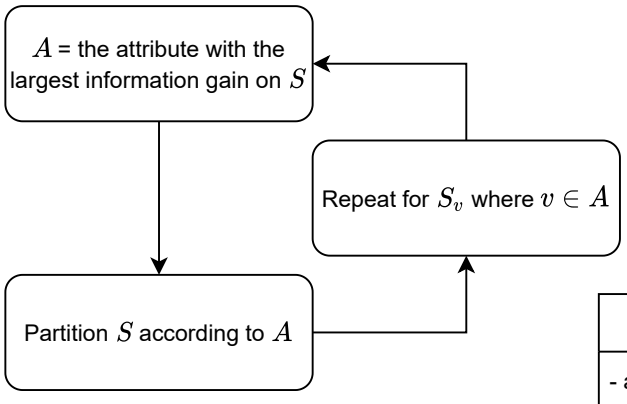


Outlook	Temp	Hum	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes

Outlook	Temp	Hum	Wind	Play
Overcast	Hot	High	Weak	Yes
Overcast	Cool	Normal	Strong	Yes

Outlook	Temp	Hum	Wind	Play
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Rain	Mild	Normal	Weak	Yes
Rain	Mild	High	Strong	No

ID3: a greedy algorithm for computing decision trees.



Notes about ID3:

- does not always create the optimal tree
- can get stuck on local maxima (does not backtrack)
- its hypothesis space (set of all trees) is complete (contains the optimal tree)
- has a preference for short trees

Occam's Razor:

Do not make things more complicated than necessary *(i.e. prefer simpler models over more complex models, complex models have a higher chance to overfit the data)*.

Missing Values:

- assign most common value
- posterior class probability

Oblique Decision Trees

Decision trees where data is split according to multiple attributes (e.g. a mathematical function like $x + y < 1$). These trees are very computationally heavy to optimize.

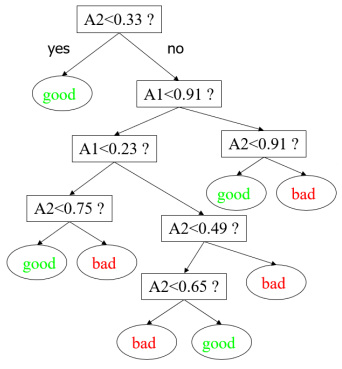
$$\text{SplitInfo}(S, A) = -\sum_{v \in A} \frac{|S_v|}{|S|} \log_2\left(\frac{|S_v|}{|S|}\right)$$

$$\text{GainRatio}(S, A) = \frac{\text{Gain}(S, A)}{\text{SplitInfo}(S, A)}$$

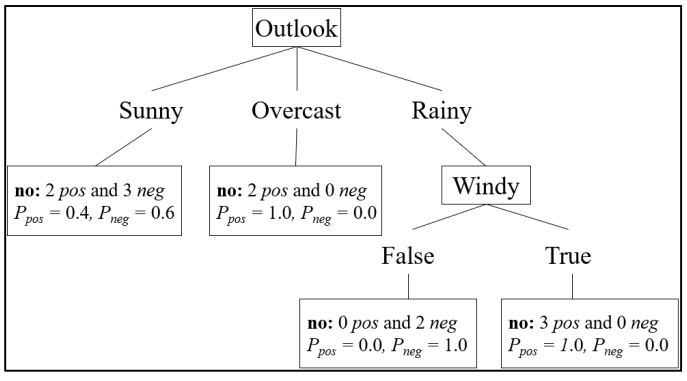
Continuous Attributes

For continuous attributes (e.g. temperature), turn the values into 'buckets', then use these buckets as attributes. Split where the information gain is the highest. Because of this feature, decision trees are **non-linear classifiers.**

Temp.	Play
64	Yes
65	No
68	Yes
69	Yes
70	Yes
71	No
72	No
72	Yes
75	Yes
75	Yes
80	No
81	Yes
83	Yes
85	No



Posterior Class Probabilities:

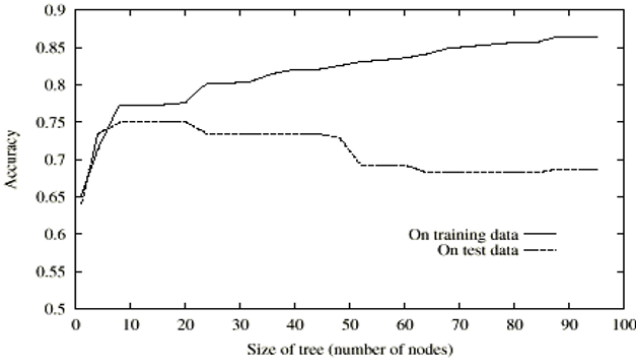


Discrete Attributes With Many Values

The problem is that they fragmant the data too quickly, leaving insufficient data at the next level. A solution to this would be to only perform binary splits, or use *GainRatio* instead of information gain.

Overfitting

Given a hypothesis space H, a hypothesis $h \in H$ is overfitting the data if there exists another hypothesis $h' \in H$ such that h has smaller error on the training data, but h' has a smaller error over the data distribution. A result of this is that the tree will be very complicated, and fewer instances will be associated with leaf notes.



Pre-Pruning

Stop growing the tree earlier, before it reaches the point where it perfectly classifies the training data. A common way of implementing this is introducing a minimal size for S_v. e.g. "this category must have at least 5 instances to partition it, otherwise we just use the most common outcome."

Reduced-Error (post) Pruning

First split the data into a **growing set** and a **validation set.** Then grow the subtree (allowing it to overfit on the growing set) and start pruning:

1) For every non-leaf node d, remove the subtree with root d (turning node d into a leaf)

2) Calculate the accuracy of this new tree (with the subtree removed), on the validation set

3) Repeat (keep pruning) until further pruning is harmful (hill-climbing)

4) At the end choose the tree with the highest accuracy

Rule Post-Pruning

Convert the entire tree into a set of equivalent IF statements (e.g. IF (outlook = sunny & humidity = high) THEN play_tennis = no), sort these statements in order of (estimated) accuracy. When classifying further instances, apply these statements in order until a result is reached.